Conditional Statements

Choosing which branch runs

Gregory S. DeLozier, PhD

Breaking Changes

`if` and `else` become keywords.

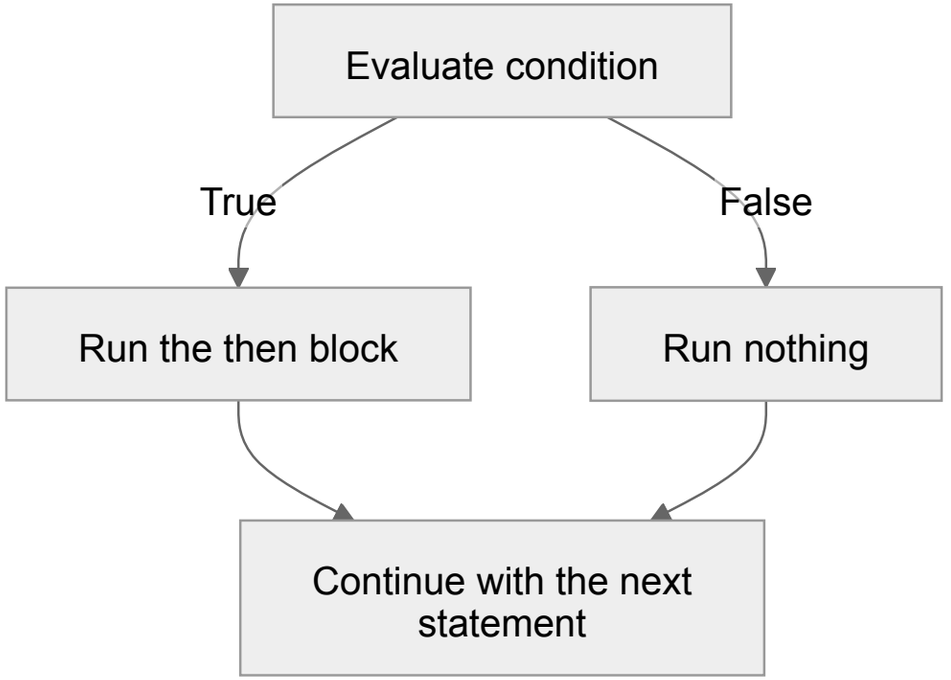
`{` and `}` are now reserved for blocks.

Only One Branch Runs

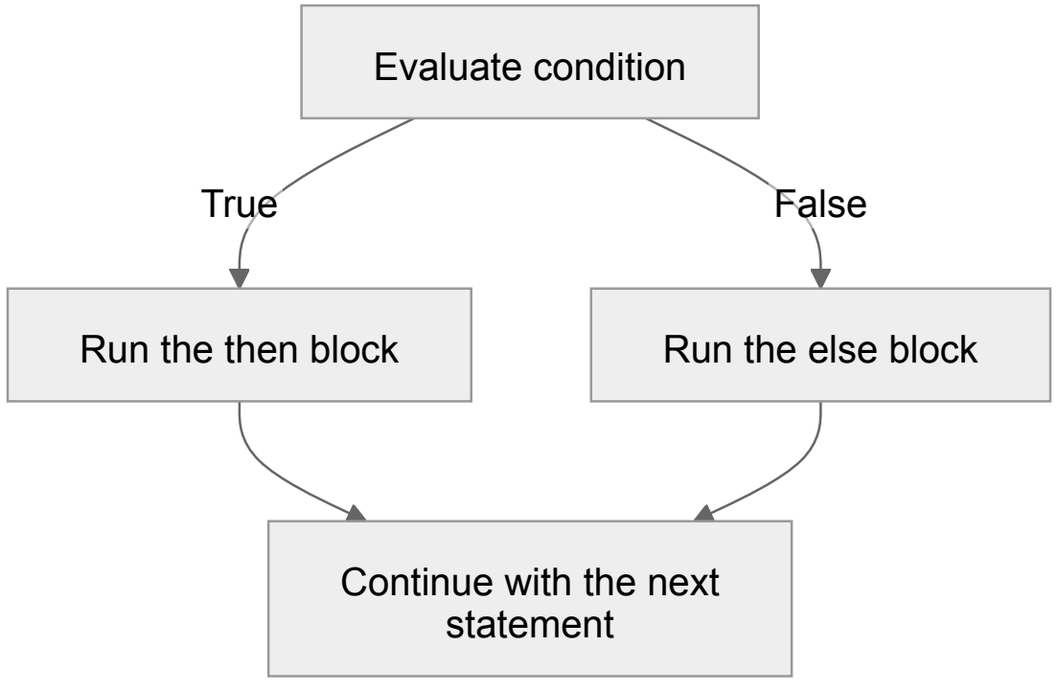
```
if (temperature > 90) {
    print("Too hot");
} else {
    print("Fine");
}
```

The condition must be a boolean.

If, No Else


Evaluate condition
True
False
Run the then block
Run nothing
Continue with the next statement

If, With Else



Exactly one branch runs, either way.

The Unselected Branch Is Never Evaluated

```
if (false) {
    exit(9);
    print(number(input()));
} else {
    print("only this branch runs");
}
```

Not skipped output. Skipped evaluation.

Every Branch Is a Block

```
if_statement ::= "if" "(" expression ")" block [ "else" block ]
block ::= "{" [ statement_list ] { ";" } "}"
```

A bare statement is never a valid branch.

Why Require Braces Everywhere

```
if (a) if (b) x=1; else y=2;
```

Which `if` does the `else` belong to?

No “else if” Shortcut

```
if (grade >= 90) {
    letter = "A";
} else {
    if (grade >= 80) {
        letter = "B";
    } else {
        letter = "C or lower";
    };
};
```

Parsing a Block

```
def parse_block(tokens):
    tokens = require(tokens, "{", "Expected '{' to start a block")
    while tokens[0]["tag"] == ";":
        tokens = tokens[1:]
    if tokens[0]["tag"] == "}":
        return {"tag": "statement_list", "statements": []}, tokens[1:]
    statements, tokens = parse_statement_list(tokens)
    tokens = require(tokens, "}", "Expected '}' to close a block")
    return statements, tokens
```

Empty Blocks Are Legal

```
if (found) { }
```

`{}`, `{ ; }`, and `{ ;;; }` are empty blocks.

A Second Caller for statement_list

```
if tokens[0]["tag"] in (None, "}"):
    break
```

A trailing semicolon before } is now allowed too.

Semicolons Around a Conditional

```
if (true) { x=1 };
y=2;
```

A semicolon after `}`, none before `else`.

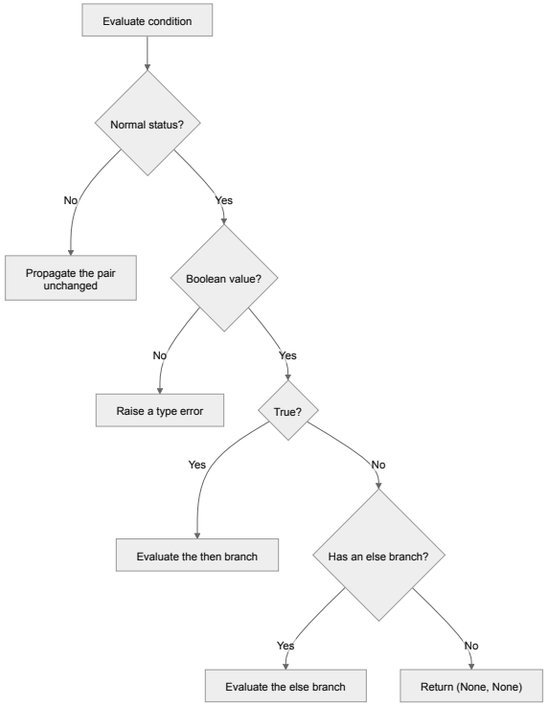
Parsing an If Statement

```
def parse_if_statement(tokens):
    tokens = require(tokens, "if", "Expected 'if'")
    tokens = require(tokens, "(", "Expected '(' after 'if'")
    condition, tokens = parse_expression(tokens)
    tokens = require(tokens, ")", "Expected ')' after if condition")
    then_block, tokens = parse_block(tokens)
    else_block = None
    if tokens[0]["tag"] == "else":
        else_block, tokens = parse_block(tokens[1:])
    return {"tag": "if", "condition": condition, "then": then_block,
            "else": else_block}, tokens
```

Evaluating a Conditional

```
if ast["tag"] == "if":
    condition, status = evaluate(ast["condition"], environment)
    if status is not None:
        return condition, status
    if type(condition) is not bool:
        raise TypeError("if condition must be a boolean expression")
    if condition:
        return evaluate(ast["then"], environment)
    if ast["else"] is not None:
        return evaluate(ast["else"], environment)
    return None, None
```

The Full Decision



Status Still Escapes Nested Blocks

```
x = 1;
if (true) {
    if (true) {
        exit(7);
    };
};
x = 2;
```

Result: `(7, "exit")`. `x` stays 1.

Testing the Contract

Single- and two-branch conditionals, nested conditionals, multi-statement blocks.

Unselected branch never touches input, exit, or assert.

Status escapes nested blocks and stops later statements.

Parser rejects unbraced bodies; accepts braced single-statement and nested blocks.

Questions for Thought

Why must the unselected branch be skipped entirely, not evaluated and discarded?

What would break if `if` consumed a status from its own selected branch?

What would adding an “else if” shortcut change about how `else` attaches?

If blocks later introduced their own scope, what would have to change here?